STRUCTML HW 2

Zina Assi

213813165

Zina.assi@campus.technion.ac.il

Abed-al-rahman Badran

325424752

badran.abed@campus.technion.ac.il

Q2 :

After building the graph, report the following graph statistics:

a. number of nodes
b. number of edges
c. Min, max, avg and median of node degrees.

a. Number of nodes: 4,121,747

b. Number of edges: 5,642,180 undirected (each PK-FK edge is stored in both directions, so 11,284,360 directed edges)

c. Node degree: min = 0, max = 68,018, avg = 2.74, median = 2

Q4 :

in 4 different ways:

a. The binary label in the prediction task (contribution) – positive and negative tuples have different colors.
b. The degree of the node (0, 1-10, 11-100, >100)..
c. The number of posts each user has - above 2 posts, below 2 posts.
d. Badges - does the user have at least 1 badge.

Add all 16 figures to your report with matching captions. Arrange them in a 4x4 structure for convenient comparison (one row for each embedding method, column for each coloring). Describe any structure or pattern that you see emerging from these visualizations.

Patterns we see:

The four colorings are strongly correlated (label, degree, posts, badges all measure how active a user is), so within each row the same regions light up. The badges coloring is the least informative since almost everyone has a badge.

Kernel PCA: the clearest structure, many small well separated clusters that match the colorings, the dense cluster on the right is high-degree contribution=1 users, the lower clusters are inactive contribution=0 users. Sharpest label separation.

Laplacian eigenmaps: now shows a clear radial/spiral structure organized by node degree (the degree coloring forms a visible gradient, highest-degree users toward the center). The labels are still fairly mixed, with positives leaning to the higher-degree regions. So it captures connectivity well but the label only through the degree-activity correlation.

Node2Vec: a radial activity pattern, low-degree contribution=0 users in the middle, higher-degree contribution=1 users toward the outside. Visible but soft.

Metapath2Vec: a dense central cluster of very active users, surrounded by a diffuse cloud of less active users where labels mix.

Q5 :

The best model in assignment 1 was XGBoost (max_depth=4, n_estimators=200, learning_rate=0.1, scale_pos_weight for the class imbalance), so we use it as the classifier on each embedding.

Method	**Accuracy**	**ROC AUC**	**Precision**	**Recall**	**F1**
Kernel PCA	0.819/0.868	0.915/0.868	0.197/0.139	0.853/0.717	0.320/0.233
Laplacian eigenmaps	0.826/0.821	0.883/0.779	0.193/0.094	0.782/0.622	0.310/0.163
Node2Vec	0.848/0.853	0.876/0.773	0.207/0.104	0.722/0.553	0.322/0.175
Metapath2Vec	0.711/0.706	0.778/0.706	0.112/0.055	0.690/0.586	0.192/0.101

Q6 :

The best performing embedding in Q5 (by validation AUC) was Kernel PCA, so the Emb⊕Emb baseline is Kernel PCA ⊕ Kernel PCA. Added rows to the table:

Method	**Accuracy**	**ROC AUC**	**Precision**	**Recall**	**F1**
Kernel PCA ⊕ Node2Vec	0.832/0.880	0.922/0.862	0.210/0.150	0.855/0.703	0.337/0.247
Kernel PCA ⊕ Laplacian eigenmaps	0.840/0.885	0.927/0.859	0.219/0.152	0.859/0.677	0.350/0.248
Kernel PCA ⊕ Metapath2Vec	0.819/0.870	0.916/0.868	0.198/0.142	0.856/0.717	0.321/0.237
Node2Vec ⊕ Metapath2Vec	0.851/0.855	0.878/0.774	0.211/0.104	0.724/0.548	0.327/0.175
Kernel PCA ⊕ Kernel PCA (baseline)	0.819/0.868	0.915/0.868	0.197/0.139	0.853/0.717	0.320/0.233

Q7 :

7. Based on your answers to questions 4-6, answer for each embedding method (including the concatenated embeddings): how well did the embedding capture differences between the two populations? Discuss the results: which embedding methods led to the best predictive performance?

Base embeddings:

Kernel PCA: captured the difference best. In the t-SNE plot the positive and negative tuples sit in clearly separated clusters, and downstream it gives the best base scores (val AUC 0.868, F1 0.233). This is because it compresses the assignment 1 features, which were engineered exactly for this task, so the task signal is already in its input.

Laplacian eigenmaps and Node2Vec: essentially tied as the best graph embeddings. Laplacian is slightly ahead on AUC (0.779 vs 0.773) and Node2Vec slightly ahead on F1 (0.175 vs 0.163). Laplacian, on the raw adjacency, and shows a clear degree gradient in its t-SNE plot,

Node2Vec shows a soft radial pattern with inactive users in the middle and active users toward the outside. Both learn how connected and active a user is from the graph alone, which correlates with the label, but with far less detail than the engineered features, so both stay well below Kernel PCA. Note that their high accuracy (around 0.82 to 0.85) hides a lower recall, so they miss many positive users.

Metapath2Vec: captured the difference weakest (val AUC 0.706, F1 0.101). It represents the very active users well (the dense central cluster in its plot, mostly contribution=1) but barely learns anything about the many low activity users, because the user-post-comment walks rarely visit them and never see votes or badges.

Concatenated embeddings:

Kernel PCA ⊕ Node2Vec: about as good as Kernel PCA. The AUC is slightly lower (0.862) but it gives a high F1 (0.247) and accuracy (0.880). So the graph adds a small improvement to the final yes/no decision, but no real improvement to the ranking of users.

Kernel PCA ⊕ Laplacian eigenmaps: the best F1 (0.248) and accuracy (0.885) in the whole table, but the AUC (0.859) is slightly below Kernel PCA alone. So the Laplacian part helps a little at the decision threshold but does not improve the ranking.

Kernel PCA ⊕ Metapath2Vec: same AUC as Kernel PCA alone (0.868), with a small F1 gain (0.237).

Node2Vec ⊕ Metapath2Vec: same as Node2Vec alone (val AUC 0.774, F1 0.175). The two walk based methods capture largely the same information, so combining them adds nothing.

Emb⊕Emb (Kernel PCA ⊕ Kernel PCA): identical to Kernel PCA alone (val AUC 0.868, F1 0.233). This is the expected baseline result: doubling the input columns without new information changes nothing. It tells us that any improvement from a real combination must come from new information, not from the larger input size.

Discussion, which methods led to the best predictive performance:

The best predictive performance came from Kernel PCA: val AUC 0.868 alone (tied with Kernel PCA ⊕ Metapath2Vec), and the Kernel PCA ⊕ Laplacian combination is best if we judge by F1 (0.248) and accuracy (0.885). The overall picture: the feature based (tabular) embedding wins because it inherits the task signal from the engineered features, while the graph embeddings carry a real but much weaker activity signal. That graph signal is already contained in the assignment 1 features (which include graph features like the degree), which is why concatenating a graph embedding to Kernel PCA improves almost nothing on AUC, just like the Emb⊕Emb baseline predicts.

Q8 :

Compare the performance results to those you got in assignment 1 by generating tabular and graph features. Did any embedding method yield improved performance over using generated features?

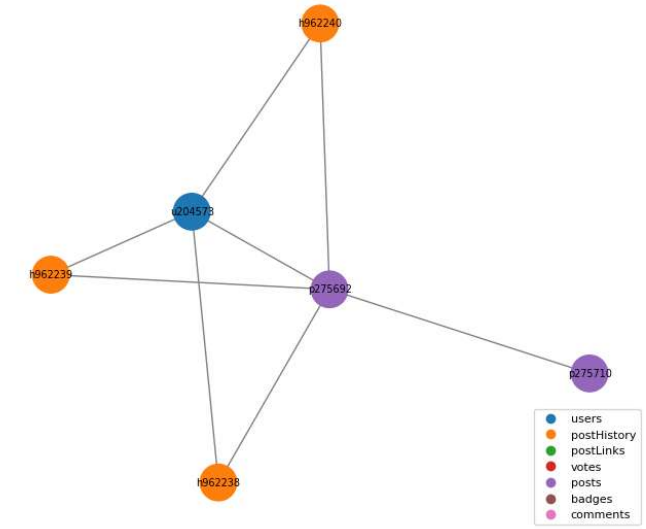
In assignment 1 the best model (XGBoost) reached val AUC 0.879 (F1 0.153) on tabular features and 0.865 (F1 0.294) with graph features. The best embedding result here is val AUC 0.868 (Kernel PCA), so no embedding beat the generated features: 0.868 is below 0.879, and the best embedding F1 (0.248, Kernel PCA + Laplacian) is below 0.294. The pure graph embeddings stayed behind (0.706 to 0.779). This is expected, since the embeddings are task-agnostic and compress everything into 128 numbers, while the hw1 features were engineered for this exact task. Kernel PCA comes closest because it is a compression of those same features. It is still notable that it loses only about 0.01 AUC, and that the pure graph embeddings recover up to 0.78 AUC with no feature engineering.

Q9 :

Qualitative analysis: sample or choose a node u (representing a user) from the graph, and create a visualization of their 2-hop neighborhood. For each visualized node v, add to the figure (possibly next to graph, if you find it more convenient): their type in the graph (i.e., the table it came from), its ID, and the embedding similarity of the node v with u, in each of the base embeddings (from the table in q5). Repeat this process for **3 different nodes u.** Describe any differences you find between the embedding methods in these visualizations.

We chose 3 users from the validation table with degree between 3 and 8, so the picture stays readable, and sampled at most 7 neighbors per node.

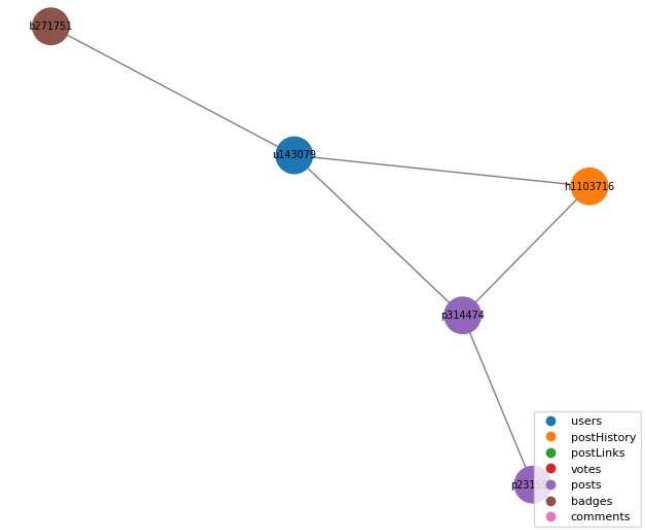
2-hop neighborhood of user 204573 (max 7 neighbors sampled per node)



type, id and cosine similarity to u per base embedding
(Kernel PCA of a non-user node = embedding of its owner user)

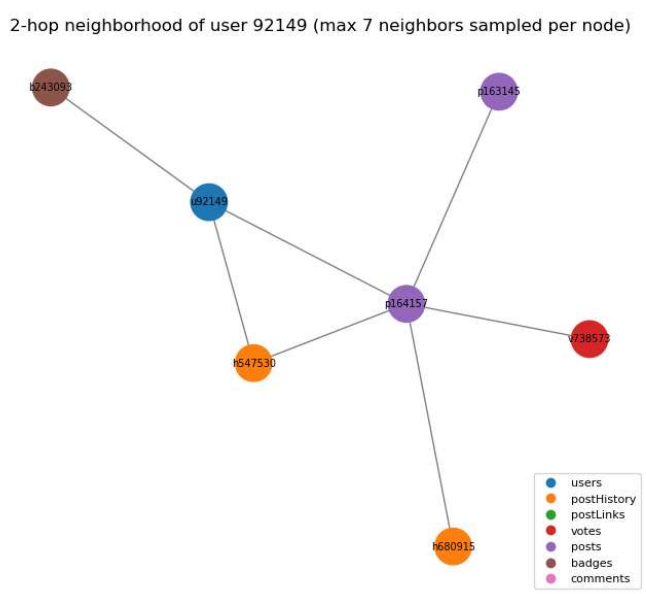
node	hop	table	id	KernelPCA	Laplacian	Node2Vec	Metapath2Vec
u204573	0	users	204573	1.00	1.00	1.00	1.00
h962238	1	postHistory	962238	1.00	1.00	0.60	-
h962239	1	postHistory	962239	1.00	1.00	0.42	-
h962240	1	postHistory	962240	1.00	1.00	0.41	-
p275692	1	posts	275692	1.00	0.35	0.49	0.19
p275710	2	posts	275710	-0.04	1.00	0.47	-0.14

2-hop neighborhood of user 143079 (max 7 neighbors sampled per node)



type, id and cosine similarity to u per base embedding
(Kernel PCA of a non-user node = embedding of its owner user)

node	hop	table	id	KernelPCA	Laplacian	Node2Vec	Metapath2Vec
u143079	0	users	143079	1.00	1.00	1.00	1.00
b271751	1	badges	271751	1.00	0.01	0.35	-
h1103716	1	postHistory	1103716	1.00	1.00	0.42	-
p314474	1	posts	314474	1.00	0.00	0.45	0.04
p231595	2	posts	231595	0.40	0.99	0.23	-0.12



type, id and cosine similarity to u per base embedding
(Kernel PCA of a non-user node = embedding of its owner user)

node	hop	table	id	KernelPCA	Laplacian	Node2Vec	Metapath2Vec
u92149	0	users	92149	1.00	1.00	1.00	1.00
b243093	1	badges	243093	1.00	0.01	0.45	-
h547530	1	postHistory	547530	1.00	1.00	0.57	-
p164157	1	posts	164157	1.00	0.00	0.53	0.10
h680915	2	postHistory	680915	0.10	1.00	0.44	-
p163145	2	posts	163145	0.07	0.01	0.37	0.12
v738573	2	votes	738573	-	1.00	0.42	-

Differences we find between the embedding methods:

Laplacian eigenmaps gives a wide range of similarities: close to 1.00 for some of the user's records (the postHistory and votes nodes) and close to 0 for others (badges and several posts, for example 0.01 for the badge and 0.00 for a post of user 92149). The values reflect the node's spectral or structural position rather than the hop distance, so they do not decrease cleanly from hop 1 to hop 2.

Node2Vec is the most informative locally: direct neighbors get moderate similarity (about 0.35 to 0.6) and 2-hop nodes get less (about 0.23 to 0.47). The similarity decreases with the hop distance, so it actually distinguishes between closer and farther nodes.

Metapath2Vec covers only users, posts and comments, so for badges, votes and postHistory nodes there is no embedding at all (NaN in the tables). Even for the user's own posts the similarity is low (0.04 to 0.2, and sometimes negative at hop 2). These are low degree users that are rarely visited by the user-post-comment walks, so their embeddings are barely trained. This matches the diffuse outer cloud of Metapath2Vec in the Q4 figure.

Kernel PCA is only defined between user nodes from the validation table, therefore a non-user node takes the embedding of its owner user. For all three users every hop-1 neighbor is the user's own content (their posts, badges, postHistory), so it maps back to u itself and gets similarity 1.00. The hop-2 nodes belong to other users, and there Kernel PCA gives a real cross-user similarity if that user is in the validation table (0.10 and 0.07 for user 92149, 0.40 for user 143079, -0.04 for user 204573) or "-" if not (the vote v738573, whose voter is outside the validation table). So Kernel PCA cannot tell a user apart from their own items (everything they own looks identical to them, 1.00), which is the opposite of the graph methods that do separate a user from their posts.

Q10 :

10. For each base embedding method you used in this assignment (Kernel PCA, Node2Vec, Laplacian Eigenmaps, Metapath2Vec), explain how it instantiates the encoder-decoder paradigm (What are the decoder, loss, and proximity definitions)? As a result of the choice of these elements, what does each embedding method aim to preserve?

Kernel PCA

Decoder: $DEC(z_u, z_v) = z_u \cdot z_v$

Proximity: The RBF kernel similarity $K(x_u, x_v)$ between the feature vectors of the two tuples.

Loss: Squared (Frobenius) reconstruction error of the kernel matrix: $\sum_{u,v}(z_u \cdot z_v - K(x_u, x_v))^2$

Aims to preserve:
Pairwise feature similarity. Tuples with similar attribute values receive similar embeddings. The graph structure is not used. The encoder is a projection applied directly to the feature vectors rather than a lookup table, allowing new tuples to be embedded without retraining.

Laplacian Eigenmaps

Encoder:
Embedding lookup.

Decoder:
Classically, squared Euclidean distance: $DEC(z_u, z_v) = || z_u - z_v ||^2$

In our randomized-SVD implementation, the decoder is the inner product: $DEC(z_u, z_v) = z_u \cdot z_v$

which reconstructs adjacency matrix entries.

Proximity:
The raw (un-normalized) adjacency matrix: $S(u, v) = 1$

if nodes u and v are connected by a PK-FK edge.

Loss:
The classical objective minimizes $\sum_{u,v} S(u, v) \ || z_u - z_v ||^2$

which encourages connected nodes to have similar embeddings. In our implementation, truncated SVD provides the best rank-k approximation of the adjacency matrix.

Aims to preserve:
First-order (direct) connectivity. Nodes connected by an edge are embedded close together, preserving local graph structure.

Node2Vec

Decoder: Softmax over inner products: $p(v \mid u) \propto \exp(z_u \cdot z_v)$

Proximity: Probability that node $v$appears within a context window (size 5) around node $u$on a random walk of fixed length (24).

Loss: Cross-entropy between the decoded distribution and the node co-occurrences observed in the random walks, optimized using negative sampling (Skip-Gram objective).

Aims to preserve: Multi-hop stochastic neighborhood overlap. Nodes whose random walks visit similar regions of the graph obtain similar embeddings, even if they are not directly connected

Metapath2Vec

Decoder: Same as Node2Vec:$p(v \mid u) \propto \exp(z_u \cdot z_v)$

Proximity: Node co-occurrences on random walks constrained to follow the metapath:

User → Post → Comment → User

Loss: Same as Node2Vec: Skip-Gram objective with negative sampling.

Aims to preserve:
Schema-aware (semantic) proximity in heterogeneous graphs. Only relationships defined by the selected metapath contribute to similarity. Thus, users connected through posting and commenting become similar, while relationships through votes, badges, or post history are intentionally ignored.

Q11 :

11. Why are embedding methods usually considered as task-agnostic?

Because they are trained without the task labels. The training objective only tries to preserve some notion of proximity in the data itself (feature similarity, adjacency, walk co-occurrence) and never mentions the label we want to predict. As a result, the same embedding can be computed once and then reused as input for many different downstream tasks. The price is that nothing forces the embedding to keep exactly the information that a specific task needs.

Q12 :

12. In questions 1 and 3a you have used two different methods to handle the large size of the matrix. Explain, in high level, how each of them works. Could we have used the random_svd method for the kernelPCA in question 1, and why?

Nystroem (Q1)

Instead of computing the full $n \times n$ kernel matrix, Nystroem selects a small set of landmark points ($m = 256$). It computes kernel similarities only between all points and the landmarks, and among the landmarks themselves. The eigendecomposition of the small landmark matrix is then used to create an approximate kernel feature space. This avoids storing the huge kernel matrix while still approximating kernel PCA.

randomized_svd (Q3a)

A fast approximation of SVD. It first creates a small random sketch of the matrix, then performs SVD on this much smaller representation. Since it only requires matrix multiplications, it is efficient for very large sparse matrices such as our adjacency matrix.

Why not use randomized_svd for Kernel PCA

Kernel PCA requires the eigendecomposition of the full RBF kernel matrix. Since this matrix is extremely large and dense, it cannot be stored efficiently. Using randomized_svd would still require access to this kernel matrix, defeating the purpose. Applying randomized_svd directly to the feature matrix would perform ordinary PCA rather than kernel PCA, so it would not preserve the required RBF-based similarities.

Q13 :

13. In metapath2vec, what are the constraints on the length of the random walk compared to the length of the metapath? (Should one be longer than the other? Are there any other constraints?) And why?

The walk is generated by following the metapath relations step by step, and when it reaches the end of the metapath it repeats it cyclically.

This gives the following constraints:

The walk should be longer than the metapath, usually several times longer. A walk shorter than the metapath never completes the pattern even once, and a walk of exactly one pattern produces very few training pairs. In our setting the walk length is 24, which is 8 repetitions of the 3 edge metapath, so each walk produces many context windows.

If the walk is longer than the metapath, the metapath must be cyclic: it has to end at the same node type it starts from. Ours is user-post-comment-user, starting and ending at user. The reason: step i of the walk uses relation number i mod (metapath length), so after finishing one pass the walker must be standing at a node type from which the first relation can be applied again. A non cyclic metapath would leave the walker at a type with no legal next step.

The walk must also be at least as long as the context window (walk_length + 1 >= context_size). The training pairs are cut out of the walk using a sliding window of size context, so a walk shorter than the window produces no (center, context) pairs at all.

Q14 :

a. Steps we took and what we learned

We converted the Assignment 1 feature matrix into a fully numeric representation and compressed it using approximate Kernel PCA (Nystroem + PCA). This showed us how kernel methods can be scaled to millions of rows by using approximations instead of constructing the full kernel matrix.

We built the PK-FK graph using RelBench utilities, resulting in a graph with 4.1M nodes and 5.6M edges. We learned that RelBench provides graph construction and analysis tools that replace much of the manual graph processing required in Assignment 1.

We trained three graph embedding methods: Laplacian Eigenmaps, Node2Vec, and Metapath2Vec. This taught us the practical aspects of spectral methods (sparse matrix operations) and random-walk methods (batching, sparse gradients, GPU training), as well as how metapaths control the information available to the model.

We evaluated the embeddings using t-SNE visualization, XGBoost classification, and neighborhood analysis. Across all methods, the dominant signal captured was user activity level.

b. Tabular feature extraction vs. learning embeddings

In Assignment 1, the analyst explicitly chooses which tables to join, which aggregations to compute, and which time windows to use. As a result, every feature has a clear interpretation.

With embedding methods, the analyst instead chooses the embedding algorithm, the notion of proximity to preserve, the embedding dimension, and training hyperparameters. The resulting features are learned through optimization and generally have no direct interpretation.

Advantages of embeddings:

- Require little or no manual feature engineering.
- Capture complex multi-hop relationships.
- Produce compact fixed-size representations.
- Can be reused across multiple tasks.

Disadvantages of embeddings:

- Difficult to interpret.
- May capture information irrelevant to the target task.
- Require training and hyperparameter tuning.

- Often perform poorly for new or weakly connected nodes.

c. Graph embeddings vs. tabular embeddings

Kernel PCA on the Assignment 1 features achieved the best downstream performance (validation AUC = 0.868), close to the original feature set (0.879). This is because it compresses features specifically engineered for the prediction task and provides coverage for all users, including inactive ones. However, it cannot capture information that was not explicitly engineered into the features.

Graph embeddings required no feature engineering and still recovered meaningful activity and connectivity patterns. Laplacian eigenmaps and Node2Vec were the best of them, with validation AUC of 0.779 and 0.773 using only graph structure. However, most of this information was already present in the engineered features: combining graph embeddings with Kernel PCA did not improve validation AUC and only slightly increased F1 score (0.248 vs. 0.233).

Additional limitations of graph embeddings are that they provide little information for weakly connected users, ignore attribute values, and each method captures only the specific notion of proximity it was designed for. This was especially evident in Metapath2Vec, which only considers User–Post–Comment relations and achieved the lowest validation AUC (0.706).

Q15 :

15. **Packages you used:** if you used any packages other than those mentioned in this assignment specification, list them in your report, and describe what you used them for (about a sentence per package).

pandas and numpy: loading and handling the feature matrices, building the results tables.

matplotlib: all the plots (training loss curves, the t-SNE grid in Q4, the neighborhood figures in Q9).

Q16 :

Use of AI Tools

We used Claude (Claude Code by Anthropic in VSCode) for much of the coding work in this assignment, including generating initial notebook cells for graph construction, embedding training, evaluation, visualization, and drafting preliminary textual answers.

To validate the generated code, we reviewed every cell before execution, checked intermediate outputs (shapes and data types), compared graph statistics with the underlying RelBench tables, monitored training loss during optimization, and verified that downstream performance metrics were consistent with our Assignment 1 results. For uncertain implementation details, such as MetaPath2Vec behavior on dead-end walks and RelBench's key reindexing process, we consulted the package source code directly.

This validation process identified several issues, including an incorrect assumption that all task users existed in the graph. An assertion revealed that some users were created after the validation timestamp, and the code was corrected accordingly.

Overall, we estimate that AI assisted with approximately 70% of the workflow.